```
class League:
    def __init__(self, league_id, name, country):
        self.league_id = league_id
        self.name = name
        self.country = country
        self.teams = []
        self.statistics = {}

    def add_team(self, team):
        if any(t.team_id == team.team_id for t in self.teams):
            print(f"Team ID {team.team_id} already exists in the league.")
        else:
            self.teams.append(team)
            print(f"Team {team.name} created in the league {self.name}.")

    def remove_team(self, team_id):
        initial_team_count = len(self.teams)
        self.teams = [team for team in self.teams if team.team_id != team_id]
        if len(self.teams) == initial_team_count:
            print(f"Team ID {team_id} not found in the league.")
        else:
            print(f"Team ID {team_id} Removed from the league.")

    def update_league(self, name=None, country=None):
        if name is not None:
            print(f"League name changed from '{self.name}' to '{name}'")
            self.name = name
        if country is not None:
            print(f"League country changed from '{self.country}' to '{country}'")
            self.country = country
```

```
    def get_info(self):
        return {"league_id": self.league_id, "name": self.name, "country": self.country, "teams": len(self.teams)}

    def __str__(self):
        return f"League '{self.name}', {self.country}, Teams registered: {len(self.teams)}"


class Team:
    def __init__(self, team_id, name, manager):
        self.team_id = team_id
        self.name = name
        self.manager = manager

    def __str__(self):
        return f"Team '{self.name}', managed by {self.manager}"


class Statistic:
    def __init__(self, stat_id, description):
        self.stat_id = stat_id
        self.description = description
        self.data = {}  # Dictionary to store statistics data, e.g., {team_id: value}

    def update_statistics(self, team_id, value):
        self.data[team_id] = value

    def get_statistics(self):
        return self.data
```

```
class LeagueManager:
    def __init__(self):
        self.leagues = {}

    def create_league(self, league_id, name, country):
        if league_id in self.leagues:
            print(f"League ID {league_id} already exists.")
        else:
            self.leagues[league_id] = League(league_id, name, country)
            print(f"League: '{name}',League ID: '{league_id}' Created")

    def delete_league(self, league_id):
        if league_id in self.leagues:
            del self.leagues[league_id]
            print(f"League of ID: '{league_id}' Deleted")
        else:
            print(f"League ID {league_id} not found.")

    def update_league(self, league_id, name=None, country=None):
        if league_id in self.leagues:
            self.leagues[league_id].update_league(name, country)
        else:
            print(f"League ID {league_id} not found.")

    def manage_league_teams(self, league_id, team_id, action, team_name=None, manager=None):
        league = self.leagues.get(league_id)
        if league:
            if action == 'add':
                if team_name and manager:
                    league.add_team(Team(team_id, team_name, manager))
                else:
```

```
                print("Team name and manager must be provided to add a team.")
            elif action == 'remove':
                league.remove_team(team_id)
            else:
                print("Invalid action. Use 'add' or 'remove'.")
        else:
            print(f"League ID {league_id} not found.")

    def track_league_statistics(self, league_id, stat_id, description=None):
        league = self.leagues.get(league_id)
        if league:
            if stat_id not in league.statistics:
                league.statistics[stat_id] = Statistic(stat_id, description)
            return league.statistics[stat_id].get_statistics()
        else:
            print(f"League ID {league_id} not found.")
            return None

    def update_statistics(self, league_id, stat_id, team_id, value):
        league = self.leagues.get(league_id)
        if league:
            if stat_id in league.statistics:
                league.statistics[stat_id].update_statistics(team_id, value)
            else:
                print(f"Statistic ID {stat_id} not found in league {league_id}.")
        else:
            print(f"League ID {league_id} not found.")

leagues = LeagueManager()

leagues.create_league(1, "Premier League", "England")
```

```
leagues.update_league(1, "English Premier League")
leagues.delete_league(1)

leagues.create_league(2, "Indian Premier League", "India")
leagues.manage_league_teams(2, 101, 'add', "A", "Manager A")
leagues.manage_league_teams(2, 102, 'add', "B", "Manager B")
leagues.manage_league_teams(2, 101, 'remove')

leagues.create_league(3, "Premier League", "England")
leagues.manage_league_teams(3, 101, 'add', "C", "Manager A")
leagues.track_league_statistics(3, 201, "Goals Scored")
leagues.update_statistics(3, 201, 101, 10)
leagues.track_league_statistics(3, 201, "Goals Scored")
```